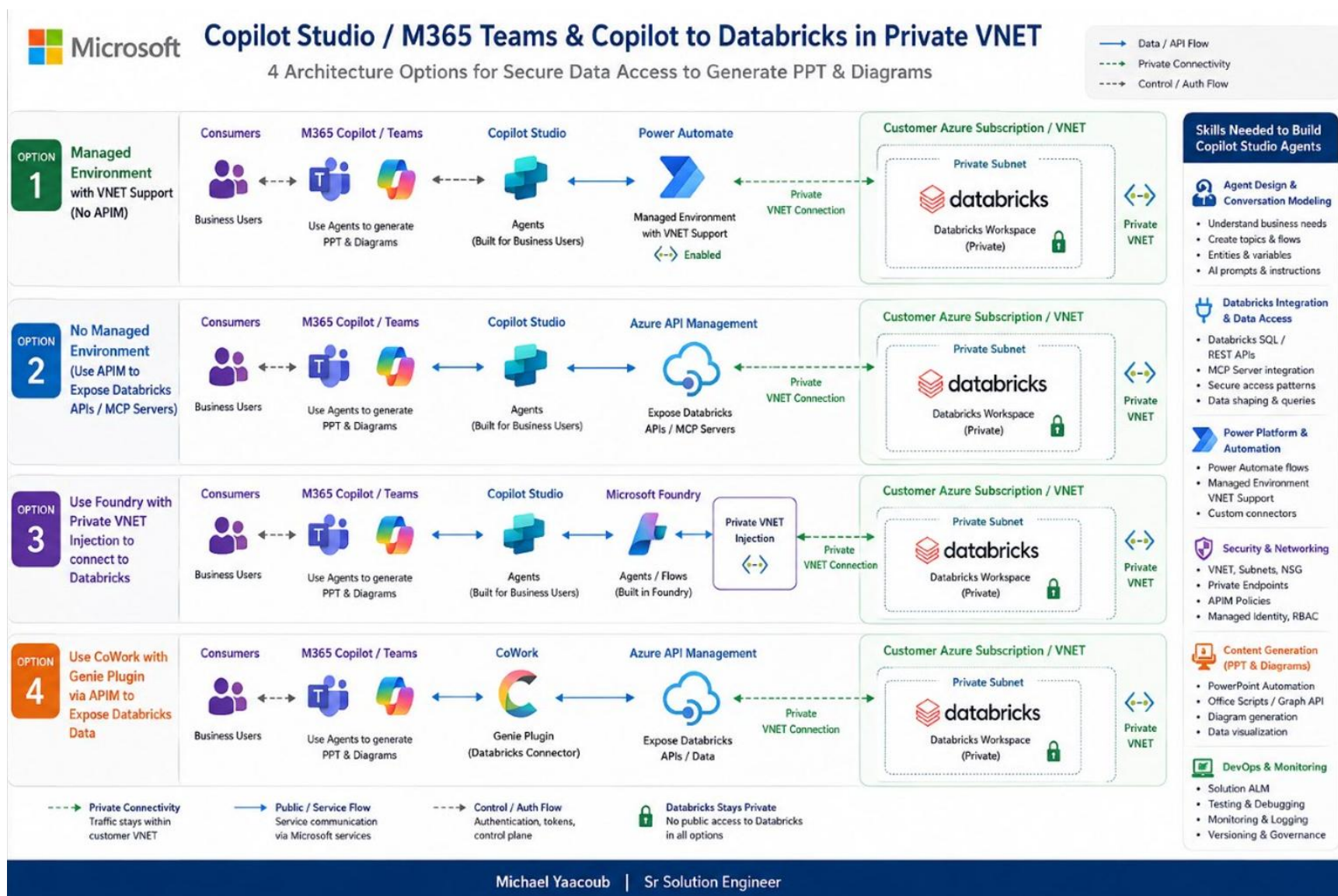




Building PPT Presentations in M365 Copilot from Databricks data in private VNET





Approach	Title & Description	Pros	Cons	Scalability – Adding Knowledge Sources	Maintainability
1. Managed Environment + VNET Support	Native Power Platform Private Connectivity — Copilot Studio agents use Power Automate in a Managed Environment with VNET Support enabled to securely reach Databricks while keeping Databricks private.	• Strong Power Platform alignment • No APIM required • Private network connectivity • Less custom infrastructure • Good fit for business-user agents	• Dependent on supported VNET-enabled connectors/connectivity patterns • Less centralized API governance than APIM • Can become more complex as integrations expand beyond supported connectors	High for supported connectors and data sources. Additional sources can be added through Power Platform connectors/flows, but each source may require its own connectivity configuration.	High initially; Power Platform handles much of the integration. Governance becomes more important as the number of flows/connectors grows.
2. API Management + Databricks APIs/MCP	API-First Integration with APIM — Databricks remains private while Azure API Management exposes governed APIs/MCP endpoints to Copilot Studio agents.	• Centralized API gateway • Authentication, authorization and throttling • Excellent API governance • Reusable APIs	• Adds APIM infrastructure and operational overhead • Requires API/MCP design and lifecycle management • Network architecture is more involved	Very High. New knowledge sources can be exposed as additional APIs/MCP tools without redesigning the agent architecture. Excellent for an enterprise with many	Very High when APIs are standardized. APIM becomes the central integration and governance layer. Requires API versioning,



Approach	Title & Description	Pros	Cons	Scalability – Adding Knowledge Sources	Maintainability
		across many agents • Strong MCP/API integration model		agents and data sources.	monitoring and ownership.
3. Microsoft Foundry + Private VNET Injection	Agentic AI with Private Network Integration — Foundry hosts the more advanced agent/orchestration layer and uses private VNET injection to securely access Databricks and other private enterprise resources.	• Strongest option for sophisticated agentic workflows • Private enterprise networking • Flexible orchestration and tool integration • Good for complex reasoning/data workflows • Can support	• Higher architecture and engineering complexity • Requires Foundry expertise • More components to operate • Potentially more development effort for business-user scenarios	Very High. Particularly strong when adding multiple enterprise knowledge sources, custom tools, APIs, databases and complex agent workflows.	Medium–High. Highly flexible, but flexibility comes with greater engineering responsibility. Requires disciplined agent, tool, network and observability management.



Approach	Title & Description	Pros	Cons	Scalability – Adding Knowledge Sources	Maintainability
		broader AI engineering patterns			
4. CoWork + Genie Plugin + APIM	Business Productivity + Governed Data Access — M365/CoWork uses a Genie plugin , with APIM providing the governed interface to Databricks data while Databricks remains private.	• Business-user friendly • Good M365 productivity experience • Reuses APIM governance • Genie provides a business-oriented interface to Databricks • Good for analytics/data exploration scenarios	• Depends on CoWork/Genie capabilities and integration maturity • APIM still required • May be less flexible than Foundry for sophisticated multi-agent orchestration	High–Very High. Additional Databricks knowledge/data capabilities can be exposed through APIM and made available through the plugin architecture.	High if APIM and Genie interfaces are standardized. Centralized APIs simplify governance and lifecycle management



Recommended Positioning

Requirement	Best Fit
Simplest architecture with minimum additional infrastructure	Option 1 – Managed Environment + VNET Support
Enterprise API governance and many future knowledge sources	Option 2 – APIM + Databricks APIs/MCP
Advanced agentic AI, complex orchestration and private enterprise data	Option 3 – Foundry + Private VNET Injection
Business productivity / M365 experience centered around Databricks data	Option 4 – CoWork + Genie + APIM
Maximum reuse across multiple agents and applications	Option 2 or 3
Lowest implementation complexity	Option 1
Most flexible long-term AI architecture	Option 3
Strongest centralized API governance	Option 2 / 4

Overall Recommendation

For an enterprise architecture where **Databricks must remain private** and the goal is to grow from a single PPT-generation agent into **multiple business agents consuming many knowledge sources**, I would position **Option 2 (APIM + Databricks APIs/MCP)** as the strongest general-purpose enterprise pattern.



Option 1 is attractive when the organization wants the simplest Power Platform-native implementation and the required connectors are supported.

Option 3 is the strategic choice when the roadmap includes sophisticated agentic workflows, multiple specialized agents, custom tools, complex orchestration, and broader private enterprise data access.

Option 4 is particularly compelling when the primary experience is **M365/CoWork business productivity with Databricks as the governed data/analytics source**.

For the architecture diagram, I would also add a visual “**Recommended for**” badge to each lane and a bottom-level decision framework: **Simplicity → Option 1 | Enterprise API Scale → Option 2 | Advanced Agentic AI → Option 3 | M365/Data Productivity → Option 4**.